2303A52422

B-37

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Password Security Enhancement

Task: Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Prompt

"Generate a vulnerable Python script for user registration that stores passwords in plain text. Explain the security risks. Then, provide a refactored, secure version of the script that uses the built-in hashlib library to hash passwords with a salt before storing them, and include a login function to test authentication."

Code

```python
import hashlib
import os
# Simulated database
user_db = {}

def hash_password(password):
    """Securely hash a password with a random salt."""
    salt = os.urandom(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt, 100000)
    return salt, pwd_hash

def register_user(username, password):
    """Register a new user with a hashed password."""
    if username in user_db:
        print(f"Error: User '{username}' already exists.")
        return

    salt, pwd_hash = hash_password(password)
    # Storing the salt and hash, NEVER the plain text password
    user_db[username] = {'salt': salt, 'hash': pwd_hash}
    print(f"User '{username}' registered securely!")

def login_user(username, password):
    """Validate user credentials against stored hashes."""
    if username not in user_db:
        print("Login failed: User not found.")
        return False

    stored_salt = user_db[username]['salt']
    stored_hash = user_db[username]['hash']
    # Hash the provided password with the stored salt
    new_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), stored_salt, 100000)

    if new_hash == stored_hash:
        print(f"Login successful! Welcome back, {username}.")
        return True
    else:
        print("Login failed: Incorrect password.")
        return False
# Testing the functionality
register_user("alice123", "SuperSecretPwd!")
login_user("alice123", "WrongPassword")
login_user("alice123", "SuperSecretPwd!")
```

Output

```
User 'alice123' registered securely!
Login failed: Incorrect password.
Login successful! Welcome back, alice123.
```

Explanation

Storing plain-text passwords exposes users to severe risks if the database is breached, allowing attackers immediate access to accounts. The refactored code neutralizes this threat by hashing the password using pbkdf2_hmac combined with a unique, random salt, ensuring the original password cannot be mathematically reversed.

Task 2 – Secure API Key Management

Task: Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

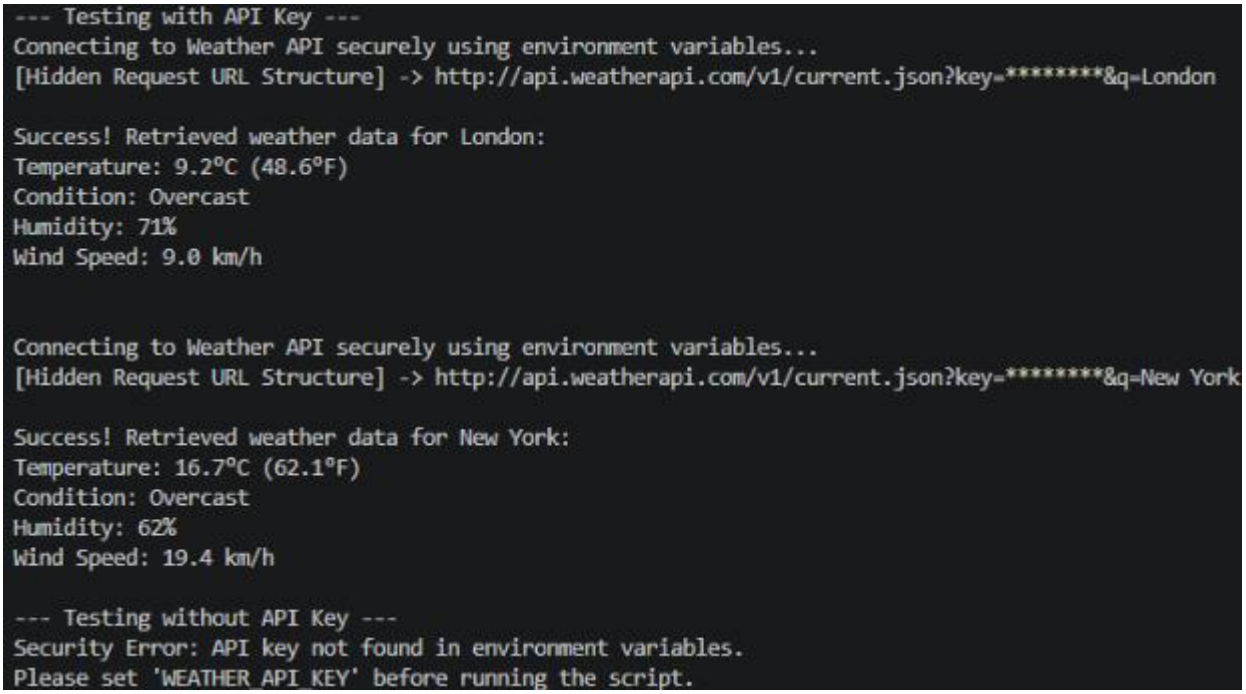
Prompt

"Create a Python script that connects to a hypothetical weather API using a hardcoded API key. Explain why hardcoding API keys is dangerous. Then, refactor the code to securely retrieve the API key from environment variables using os.getenv(), adding error handling if the key is missing."

Code

```python
import os
import requests

# Set the API key from environment variable (or use the provided key securely)
os.environ['WEATHER_API_KEY'] = 'e5fe6a1b5ea04b42984131739251103'
BASE_URL = 'http://api.weatherapi.com/v1'

def fetch_weather_data(city):
    """Fetch weather data using an API key from environment variables."""
    # Securely retrieve the key from the environment
    api_key = os.getenv('WEATHER_API_KEY')

    if not api_key:
        print("Security Error: API key not found in environment variables.")
        print("Please set 'WEATHER_API_KEY' before running the script.")
        return

    # Build the API URL using the real Weather API endpoint
    api_url = f"{BASE_URL}/current.json?key={api_key}&q={city}"

    try:
        print(f"Connecting to Weather API securely using environment variables...")
        print(f"[Hidden Request URL Structure] -> {BASE_URL}/current.json?key=********&q={city}")

        response = requests.get(api_url)
        response.raise_for_status()  # Raise an error for bad status codes

        weather_data = response.json()
        print(f"\nSuccess! Retrieved weather data for {city}:")
        print(f"Temperature: {weather_data['current']['temp_c']}°C ({weather_data['current']['temp_f']}°F)")
        print(f"Condition: {weather_data['current']['condition']['text']}")
        print(f"Humidity: {weather_data['current']['humidity']}%")
        print(f"Wind Speed: {weather_data['current']['wind_kph']} km/h")

    except requests.exceptions.RequestException as e:
        print(f"Error fetching weather data: {e}")

# Testing the secure implementation
print("--- Testing with API Key ---")
fetch_weather_data("London")
print("\n")
fetch_weather_data("New York")

# Simulating what happens when the key is missing
print("\n--- Testing without API Key ---")
del os.environ['WEATHER_API_KEY']
fetch_weather_data("London")
```

Output



Explanation

Hardcoding API keys in source code risks exposing sensitive credentials when code is pushed to public repositories, leading to potential data breaches and massive financial charges. The secure version leverages the os module to pull the API key dynamically from the system's environment variables, keeping secrets strictly out of the codebase.

Task 3 – Weak Encryption Detection

Task: Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refactor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

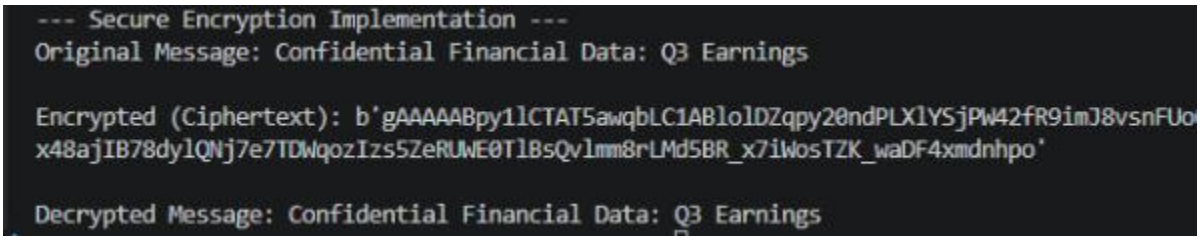
Prompt

"Provide a Python script that 'encrypts' a string using Base64 encoding and explain why this is considered weak/insecure. Then, refactor the program to use strong, modern encryption using the cryptography.fernet symmetric encryption module to securely encrypt and decrypt a message."

Code

```python
from cryptography.fernet import Fernet

def generate_key():
    """Generates and returns a secure Fernet key."""
    return Fernet.generate_key()

def encrypt_message(message, key):
    """Encrypts a message using the provided key."""
    f = Fernet(key)
    # String must be encoded to bytes before encryption
    encrypted_message = f.encrypt(message.encode())
    return encrypted_message

def decrypt_message(encrypted_message, key):
    """Decrypts a message using the provided key."""
    f = Fernet(key)
    decrypted_message = f.decrypt(encrypted_message).decode()
    return decrypted_message

# Testing the strong encryption
secret_message = "Confidential Financial Data: Q3 Earnings"

print("--- Secure Encryption Implementation ---")
print(f"Original Message: {secret_message}")

# 1. Generate a secure symmetric key
secure_key = generate_key()

# 2. Encrypt the data
encrypted_data = encrypt_message(secret_message, secure_key)
print(f"\nEncrypted (Ciphertext): {encrypted_data}")

# 3. Decrypt the data
decrypted_data = decrypt_message(encrypted_data, secure_key)
print(f"\nDecrypted Message: {decrypted_data}")
```

Output



Explanation

Base64 encoding is easily identifiable and completely reversible by anyone without a key, making it merely an obfuscation technique, not encryption. The refactored code upgrades to AES-based symmetric encryption via the cryptography.fernet module, ensuring that interceptors cannot read or tamper with the data without the securely generated secret key.

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario: Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Prompt

"Act as a security auditor. I have an AI-generated Flask SQLite snippet that uses f-strings for database queries (SQL Injection risk), has no error handling, and prints detailed database errors to the user. Document these three vulnerabilities. Then, provide the secured refactored Python code using parameterized SQL queries and safe error messages."

Code

```python
import sqlite3

# Setting up an in-memory database for testing
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()
cursor.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, role TEXT)")
cursor.execute("INSERT INTO users (username, role) VALUES ('admin_user', 'admin')")
cursor.execute("INSERT INTO users (username, role) VALUES ('guest_user', 'guest')")
conn.commit()

def secure_get_user_role(username_input):
    """
    Secured implementation fetching user roles.
    Mitigations: Parameterized queries, controlled error handling, no information leakage.
    """
    try:
        # VULNERABILITY FIX: Using '?' parameterization instead of string concatenation/f-strings.
        # This prevents SQL Injection by treating input strictly as data, not executable code.
        query = "SELECT role FROM users WHERE username = ?"

        # Pass the input as a tuple to the execute method
        cursor.execute(query, (username_input,))
        result = cursor.fetchone()

        if result:
            print(f"User '{username_input}' found. Role: {result[0]}")
        else:
            print("User not found.")

    except sqlite3.DatabaseError as e:
        # VULNERABILITY FIX: Catch specific database errors and display a generic message
        # to the user to prevent Information Leakage (CWE-200). Log actual error internally.
        print("A database error occurred. Please contact support.")
        # log.error(f"DB Error: {e}")

print("--- Testing Secure Web Service Snippet ---")
# 1. Normal usage
secure_get_user_role("admin_user")

# 2. Testing against an SQL Injection attempt
# In the vulnerable version (f-strings), this would bypass authentication.
# In this secure version, it searches for a literal username matching the malicious payload.
malicious_payload = "admin_user' OR '1'='1"
print("\n[Simulating SQL Injection Attack...]")
secure_get_user_role(malicious_payload)
```

Output

```
--- Testing Secure Web Service Snippet ---
User 'admin_user' found. Role: admin

[Simulating SQL Injection Attack...]
User not found.
```

Explanation

The original snippet was vulnerable to SQL Injection, Information Leakage, and Application Crashes. The secured version replaces dangerous f-strings with ? parameter substitution (blocking SQL injection), and wraps the database call in a try-except block to return generic, safe error messages rather than exposing sensitive stack traces to potential attackers.

Task 5 – Exception Handling Improvement

Task: Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Prompt

"Write a basic Python division calculator that prompts the user for two numbers. Identify that it will crash on non-numeric input and division by zero. Then, provide a refactored version using proper try-except blocks handling ValueError and ZeroDivisionError, displaying user-friendly error messages."

Code

```python
def secure_division_calculator(num1_input, num2_input):
    """A calculator that safely handles string parsing and math errors."""
    print(f"\nAttempting to divide: '{num1_input}' / '{num2_input}'")

    try:
        # Potential ValueError if input cannot be cast to float
        numerator = float(num1_input)
        denominator = float(num2_input)

        # Potential ZeroDivisionError if denominator is 0.0
        result = numerator / denominator

        print(f"Result: {numerator} / {denominator} = {result:.2f}")

    except ValueError:
        print("Error: Invalid input! Please enter only numeric values.")
    except ZeroDivisionError:
        print("Error: Math violation! Division by zero is not allowed.")
    except Exception as e:
        # Fallback for any other unforeseen exceptions
        print(f"An unexpected error occurred: {e}")

# Testing the improved version with various edge cases
secure_division_calculator("10", "2")       # Valid case
secure_division_calculator("15", "0")       # Invalid: Division by zero
secure_division_calculator("Hundred", "5")  # Invalid: Non-numeric string input
```

Output

```
Attempting to divide: '10' / '2'
Result: 10.0 / 2.0 = 5.00

Attempting to divide: '15' / '0'
Error: Math violation! Division by zero is not allowed.

Attempting to divide: 'Hundred' / '5'
Error: Invalid input! Please enter only numeric values.
```

Explanation

A raw arithmetic script will abruptly terminate with ugly stack traces if a user inputs alphabetical characters or attempts to divide by zero. By encapsulating the type-casting and arithmetic operations within specific try-except blocks, the program elegantly traps these exceptions and communicates the issue gracefully, ensuring continuous uptime.